

入机的自我修养

刚好卓中卓今年的综合设计就是这方面的，然后第一节课做的也是这个，所以就直接拿来用了

首先训练一个模型，我们需要准备好数据集，这里是MNIST数据集，我们直接用torchvision中的datasets.MNIST就可以获取到数据集，我们手动指定SEED。我们还需要把数据集分成训练集，测试集和验证集。

然后使用dataloader来加载数据集

所以数据集加载与预处理的代码就是data.py

```
import torch
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms

SEED = 20260901
BATCH_SIZE = 32

transform = transforms.ToTensor()

full_train_dataset = datasets.MNIST(
    root = "./data",
    train = True,
    transform = transform,
    download = True,
)

test_dataset = datasets.MNIST(
    root = "./data",
    train = False,
    transform = transform,
    download = True,
)

split_generator = torch.Generator().manual_seed(SEED)

train_dataset, val_dataset = random_split(
    full_train_dataset,
    lengths = [55000, 5000],
    generator = split_generator,
)

loader_generator = torch.Generator().manual_seed(SEED)

train_loader = DataLoader(
    train_dataset,
    batch_size = BATCH_SIZE,
    shuffle = True,
    num_workers = 0,
    generator = loader_generator,
)
```

```

val_loader = DataLoader(
    val_dataset,
    batch_size = BATCH_SIZE,
    shuffle = False,
    num_workers = 0,
)

test_loader = DataLoader(
    test_dataset,
    batch_size = BATCH_SIZE,
    shuffle = False,
    num_workers = 0,
)

print("Train samples:", len(train_dataset))
print("Validation samples:", len(val_dataset))
print("Test samples:", len(test_dataset))

images, labels = next(iter(train_loader))

print("Image batch shape:", images.shape)
print("Label batch shape:", labels.shape)
print("Image dtype:", images.dtype)
print("Label dtype:", labels.dtype)
print("Pixel min:", images.min().item())
print("Pixel max:", images.max().item())
print("First 10 labels:", labels[:10].tolist())

```

数据集准备好了，我们来构建模型，这就是一个搭积木的过程，我这里使用了5个卷积层和2个池化层(参数量比较小的同时，保证精度是够用的)，然后写好权重的初始化和前向传播的逻辑

下面是model.py

```

import torch
from torch import nn
import torch.nn.functional as F

class MNISTMicroCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1,
bias=False)
        self.bn1 = nn.BatchNorm2d(32)          # 修正: 32

        self.conv2 = nn.Conv2d(32, 32, kernel_size=3, stride=1, padding=1,
groups=16, bias=False)
        self.bn2 = nn.BatchNorm2d(32)

        self.pool1 = nn.MaxPool2d(2, 2)

```

```

        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1,
groups=16, bias=False)
        self.bn3 = nn.BatchNorm2d(64)

        self.conv4 = nn.Conv2d(64, 64, kernel_size=3, stride=1, padding=1,
groups=16, bias=False)
        self.bn4 = nn.BatchNorm2d(64)

        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv5 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1,
bias=False)
        self.bn5 = nn.BatchNorm2d(128)

        self.gap = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(128, num_classes)

        self._initialize_weights()

    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode='fan_out',
nonlinearity='relu')
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.constant_(m.weight, 1)
                nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.constant_(m.bias, 0)

    def forward(self, x):
        x = F.relu(self.bn1(self.conv1(x)))
        x = F.relu(self.bn2(self.conv2(x)))
        x = self.pool1(x)
        x = F.relu(self.bn3(self.conv3(x)))
        x = F.relu(self.bn4(self.conv4(x)))
        x = self.pool2(x)
        x = F.relu(self.bn5(self.conv5(x)))
        x = self.gap(x)
        x = torch.flatten(x, 1)
        logits = self.fc(x)
        return logits

```

接下来就是模型训练和保存了，我们这里使用CUDA来训练，训练20轮；之所以代码中有start_epoch是之前为了分批次训练残留的，比如训练了100轮感觉效果不够，就再加100轮试试
然后就是很公式化的模型训练过程了，反向传播优化参数，一边训练一边打印准确率
下面是train.py

```

import torch
import torch.optim as optim

```

```
from tqdm import tqdm
from torch import nn as nn

from data import train_loader, val_loader, test_loader
from model import MNISTMicroCNN
from evaluate import evaluate

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = MNISTMicroCNN(num_classes=10).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

start_epoch = 1
total_epoch = 20

for epoch in range(start_epoch, total_epoch + 1):
    model.train()

    train_loss = 0.0
    train_correct = 0
    train_total = 0

    loop = tqdm(train_loader, desc=f'Epoch {epoch}/{total_epoch} [Train]')

    for images, labels in loop:
        images, labels = images.to(device), labels.to(device)

        outputs = model(images)
        loss = criterion(outputs, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_loss += loss.item() * images.size(0)

        _, predicted = torch.max(outputs, 1)

        train_total += labels.size(0)
        train_correct += (predicted == labels).sum().item()

    loop.set_postfix(loss=loss.item())

train_acc = train_correct / train_total
train_loss = train_loss / train_total

# 验证
val_loss, val_acc = evaluate(
    model,
    val_loader,
    criterion,
    device
```

```
)

print(
    f'Epoch {epoch:2d}/{total_epoch} | '
    f'Train Loss: {train_loss:.4f} | '
    f'Train Acc: {train_acc:.4f} | '
    f'Val Loss: {val_loss:.4f} | '
    f'Val Acc: {val_acc:.4f}'
)
```

其实我直接把evaluate写成专门的函数，然后放到train里面来调用了，省的再写加载模型的过程最后就是推理部分，也就是evaluate了。这里相当于训练部分关掉grad，也就是不需要反向传播了，直接评估即可。这里顺便输出一下准确率和损失。

```
import torch

def evaluate(model, val_loader, criterion, device):
    model.eval()

    val_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            val_loss += loss.item() * images.size(0)

            _, predicted = torch.max(outputs, 1)

            val_total += labels.size(0)
            val_correct += (predicted == labels).sum().item()

    val_acc = val_correct / val_total
    val_loss = val_loss / val_total

    return val_loss, val_acc
```

最终的训练结果如下

```
Epoch 20/20 | Train Loss: 0.0116 | Train Acc: 0.9962 | Val Loss: 0.0202 | Val Acc: 0.9936
```

识别率高达99.36%